

Ссылки на задания на Replit:

Задание 1.1:

<https://replit.com/@evgeniiayd/lablonskaia-levghieniia-kurs-1-ghruppa-IVT-12-LR-4-zadaniie-1#1.1-main.c>

Задание 1.2:

<https://replit.com/@evgeniiayd/lablonskaia-levghieniia-kurs-1-ghruppa-IVT-12-LR-4-zadaniie-12#main.c>

Задание 1.3:

<https://replit.com/@evgeniiayd/lablonskaia-levghieniia-kurs-1-ghruppa-IVT-12-LR-4-zadaniie-13#main.c>

Задание 1.4:

<https://replit.com/@evgeniiayd/lablonskaia-levghieniia-kurs-1-ghruppa-IVT-12-LR-4-zadaniie-14#main.c>

Задание 2.1:

<https://replit.com/@evgeniiayd/lablonskaia-levghieniia-kurs-1-ghruppa-IVT-12-LR-4-zadaniie-21#main.c>

Задание 2.2:

<https://replit.com/@evgeniiayd/lablonskaia-levghieniia-kurs-1-ghruppa-IVT-12-LR-4-zadaniie-22#main.c>

Задание 2.3:

<https://replit.com/@evgeniiayd/lablonskaia-levghieniia-kurs-1-ghruppa-IVT-12-LR-4-zadaniie-23#main.c>

Отчёт.

Тема: Комплект 1: Введение в функции.

Задание 1.1

1. Постановка задачи.

Создайте две функции, которые вычисляют факториал числа:

- функцию, которая вычисляет факториал, используя цикл;
- функцию, которая вычисляет факториал, используя рекурсивный вызов самой себя.

Продemonстрируйте работу обеих функций.

2. Математическая модель.

$$n! = 1 * 2 * 3 * \dots * (n-1) * n.$$

3. Список идентификаторов.

Имя	Тип	Смысл
factone	unsigned int	Функция нахождения факториала через цикл
facttwo	unsigned int	Функция нахождения факториала с помощью рекурсии
n	unsigned int	Переменная функций factone, facttwo и входная переменная

4. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  unsigned int factone(unsigned int n) {
5      unsigned int result = 1;
6      for (unsigned int i = 2; i <= n; ++i) {
7          result *= i;
8      }
9      return result;
10 }
11
12 unsigned int facttwo(unsigned int n) {
13     if (n == 0) {
14         return 1;
15     } else {
16         return n * facttwo(n - 1);
17     }
18 }
19
20 int main()
21 {
22     unsigned int n;
23
24     printf("Enter n\n");
25     scanf("%d", &n);
26
27     printf("Factorial: %d\n", factone(n));
28     printf("Factorial: %d", facttwo(n));
29
30     return 0;
31 }

```

5. Результат выполнения программы.

```

Enter n
6
Factorial: 720
Factorial: 720
Process returned 0 (0x0)   execution time : 1.975 s
Press any key to continue.

```

Задание 1.2

1. Постановка задачи.

Объявите указатель на массив типа `int` и динамически выделите память для 12-ти элементов. Напишите функцию, которая поменяет значения чётных и нечётных ячеек массива.

2. Математическая модель.

```
t = arr[i];
arr[i] = arr[i + 1];
arr[i + 1] = t;
```

3. Список идентификаторов.

Имя	Тип	Смысл
swap_even_odd	void	Функция, меняющая местами четные и нечетные элементы
i	int	Счётчик цикла
n	int	Мощность массива
*arr	int	Указатель на массив
t	int	Промежуточная переменная
print_array	void	Функция, выводящая массив

4. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void swap_even_odd(int* arr, int n) {
5      for (int i = 0; i < (n - 1); i += 2) {
6          int t = arr[i];
7          arr[i] = arr[i + 1];
8          arr[i + 1] = t;
9      }
10 }
11
12 void print_array(int *arr, int n) {
13     for (int i = 0; i < n; i++) {
14         printf("%d ", arr[i]);
15     }
16 }
17
18 int main() {
19     int* arr = (int*)malloc(12 * sizeof(int));
20     for (int i = 0; i < 12; i++) {
21         arr[i] = i + 1;
22     }
23
24     int n = 12;
25
26     printf("Original array:\n");
27     print_array(arr, n);
28     swap_even_odd(arr, n);
29     printf("\nSwapped array:\n");
30     print_array(arr, n);
31
32     return 0;
33 }
```

5. Результат выполнения программы.

```
Original array:
1 2 3 4 5 6 7 8 9 10 11 12
Swapped array:
2 1 4 3 6 5 8 7 10 9 12 11
Process returned 0 (0x0)   execution time : 0.120 s
Press any key to continue.
```

Задание 1.3

1. Постановка задачи.

Создать две основные функции:

- функцию для динамического выделения памяти под двумерный динамический массив типа double — матрицу;
- функцию для динамического освобождения памяти под двумерный динамический массив типа double — матрицу.

Создать две вспомогательные функции:

- функцию для заполнения матрицы типа double;
- функцию для распечатки этой матрицы на экране.

Продемонстрировать работу всех этих функций в своей программе.

2. Математическая модель.

-

3. Список идентификаторов.

Имя	Тип	Смысл
allocate_matrix	double**	Функция выделения памяти под матрицу
rows	int	Количество строк
cols	int	Количество столбцов
i	int	Счётчик цикла
free_matrix	void	Функция освобождения памяти, выделенной для матрицы

fill_matrix	void	Функция заполнения матрицы
print_matrix	void	Функция вывода матрицы

4. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  double** allocate_matrix(int rows, int cols) {
5      double** matrix = (double**) malloc(rows * sizeof(double*));
6      for (int i = 0; i < rows; i++) {
7          matrix[i] = (double*) malloc(cols * sizeof(double));
8      }
9      return matrix;
10 }
11
12 void free_matrix(double** matrix, int rows) {
13     for (int i = 0; i < rows; i++) {
14         free(matrix[i]);
15     }
16     free(matrix);
17 }
18
19 void fill_matrix(double** matrix, int rows, int cols) {
20     for (int i = 0; i < rows; i++) {
21         for (int j = 0; j < cols; j++) {
22             printf("Enter an element [%d][%d]: ", i, j);
23             scanf("%lf", &matrix[i][j]);
24         }
25     }
26 }
27
28 void print_matrix(double** matrix, int rows, int cols) {
29     for (int i = 0; i < rows; i++) {

```

```

30         for (int j = 0; j < cols; j++) {
31             printf("%.2f ", matrix[i][j]);
32         }
33         printf("\n");
34     }
35 }
36
37 int main(void) {
38     int rows, cols;
39     printf("Enter the amount of rows and cols: ");
40     scanf("%d%d", &rows, &cols);
41
42     double** matrix = allocate_matrix(rows, cols);
43
44     fill_matrix(matrix, rows, cols);
45
46     printf("Matrix:\n");
47     print_matrix(matrix, rows, cols);
48
49     free_matrix(matrix, rows);
50
51     return 0;
52 }

```

5. Результат выполнения программы.

```

Enter the amount of rows and cols: 2 2
Enter an element [0][0]: 3
Enter an element [0][1]: 6
Enter an element [1][0]: 9
Enter an element [1][1]: 12
Matrix:
3.00 6.00
9.00 12.00

Process returned 0 (0x0)   execution time : 4.546 s
Press any key to continue.

```

Задание 1.4

1. Постановка задачи.

Создать функцию, которая вычисляет векторное произведение двух векторов в декартовых координатах, используя указатели на соответствующие массивы.

2. Математическая модель.

$$\vec{a} = (a_1, a_2, a_3), \quad \vec{b} = (b_1, b_2, b_3)$$

$$\vec{a} \times \vec{b} = [\vec{a}, \vec{b}] = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} = \vec{i} \begin{vmatrix} a_2 & a_3 \\ b_2 & b_3 \end{vmatrix} - \vec{j} \begin{vmatrix} a_1 & a_3 \\ b_1 & b_3 \end{vmatrix} + \vec{k} \begin{vmatrix} a_1 & a_2 \\ b_1 & b_2 \end{vmatrix}$$

3. Список идентификаторов.

Имя	Тип	Смысл
*vector1	double	Указатель на массив координат первого вектора
*vector2	double	Указатель на массив координат второго вектора
*result	double	Указатель на массив координат результирующего вектора
i	int	Счётчик цикла
Cross_Product	void	Функция, вычисляющая векторное произведение двух векторов

4. Код программы.


```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void Cross_Product(double* vector1, double* vector2, double* result) {
5      result[0] = vector1[1] * vector2[2] - vector1[2] * vector2[1];
6      result[1] = vector1[2] * vector2[0] - vector1[0] * vector2[2];
7      result[2] = vector1[0] * vector2[1] - vector1[1] * vector2[0];
8  }
9
10 int main() {
11     double vector1[3] = {1.23, 2, 4};
12     double vector2[3] = {1.24, 2.2, 4.5};
13     double result[3];
14
15     Cross_Product(vector1, vector2, result);
16
17     for (int i = 0; i < 3; i++) {
18         printf("%.3f ", result[i]);
19     }
20
21     return 0;
22 }
23

```

5. Результат выполнения программы.

```

0.200 -0.575 0.226
Process returned 0 (0x0)   execution time : 0.071 s
Press any key to continue.

```

Задание 2.1

1. Постановка задачи.

Создайте новую программу, где с клавиатуры вводится строка некоторой длины порядка 10 латинских символов (не используйте кириллицу) в классическую строку языка C, которая имеет вид массива

`char my_string[MY_SIZE]`. `MY_SIZE` определите с помощью директивы `#define`. Значение `MY_SIZE` должно превышать длину вводимой

строки с некоторым разумным запасом. Другие строки в этой задаче

можете создавать либо также как статические массивы, либо как динамические массивы, но не забывайте освобождать от динамически

выделенную память с помощью функции

```
void free(void* ptr); .
```

Выполните следующие действия и распечатайте результаты:

1. Вычислите длину строки `my_string`, используя цикл `for` и тот факт, что в языке C такие строки имеют в конце специальный нулевой символ конца строки, представленный escape-последовательностью `'\0'` (`'...'` — это тип `char`).

2. Сделайте то же самое, что в пункте 1, но создайте указатель на начало вашей строки и используйте операцию инкремента `++`.

3. Используйте функции

`size_t strlen(const char* str);` или

`size_t strlen (const char *string, size_t maxlen);` или

`size_t strlen_s(const char *str, size_t strsz);` для получения размера строки в виде значения `size_t` (псевдоним

`unsigned int`, спецификатор форматирования — `"%zu"`). Убедитесь, что ваш компилятор явно работает с опцией `-std=c11` или с

опцией для более позднего стандарта языка для поддержки функции `strlen_s`.

4. Создайте вторую строку (второй массив) и скопируйте в неё строку `my_string`, используя функцию

`char *strcpy(char *dest, const char *src);` или

`char *strncpy (char *dest, const char *src, size_t n);`.

5. Создайте ещё две строки какого-либо размера и задайте их прямо в коде без клавиатуры. Сделайте конкатенацию этих двух строк, используя

`char *strcat(char *dest, const char *src);` или

`char *strncat(char *dest, const char *src, size_t n);`. Первую

строку трактуйте как `dest` (destination) и подберите размер этого массива с запасом.

6. Сравните две новые строки, заданные в коде строковыми литералами, используя функцию

`int strcmp(const char *lhs, const char *rhs);` или

`int strncmp (const char *s1, const char *s2, size_t n).`

7. Задайте прямо в коде строку, в которой есть только латинские

символы в верхнем и нижнем регистре. Переведите строку полностью в нижний регистр и отдельно полностью в верхний регистр. Распечатайте каждый результат отдельно. Найдите сигнатуры подходящих функций (`tolower` и `toupper`), изучив базовые однобайтовые строковые функции по ссылке <https://en.cppreference.com/w/c/string/byte>.

2. Математическая модель.

-

3. Список идентификаторов.

Имя	Тип	Смысл
<code>i</code>	<code>int</code>	Счётчик цикла
<code>my_string</code>	<code>char[15]</code>	Строка, вводимая с клавиатуры
<code>length</code>	<code>int</code>	Результат
<code>*ptr</code>	<code>char</code>	Указатель на первый элемент строки <code>my_string</code>
<code>*ptr2</code>	<code>const char</code>	Указатель на первый элемент строки <code>my_string</code>
<code>length2</code>	<code>size_t</code>	Результат
<code>second_string</code>	<code>char[15]</code>	Вводимая строка
<code>string1</code>	<code>char[20]</code>	Вводимая строка
<code>string2</code>	<code>const char[10]</code>	Вводимая строка
<code>lh</code>	<code>char[5]</code>	Вводимая строка
<code>rh</code>	<code>char[5]</code>	Вводимая строка
<code>*lhs</code>	<code>const char</code>	Указатель на строку <code>lh</code>
<code>*rhs</code>	<code>const char</code>	Указатель на строку <code>rh</code>
<code>reg[15]</code>	<code>unsigned char</code>	Вводимая строка

4. Код программы.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #define MY_SIZE 15
4  #include <string.h>
5  #include <locale.h>
6  #include <ctype.h>
7
8  int main()
9  {
10     char my_string[MY_SIZE];
11     int length = 0;
12
13     //First task
14     printf("Enter string: \n");
15     scanf("%s", &my_string);
16
17     for(int i = 0; i < 15; i++)
18         if (my_string[i] != '\0') length++;
19         else break;
20
21     printf("Length of string: %d\n", length);
22
23     //Second task
24     length = 0;
25     char* ptr = &my_string[0];
26
27     for(int i = 0; i < 15; i++)
28         if (*ptr != '\0') {
29             length++;
```

```

30     *ptr++;
31 } else break;
32
33 printf("Length of string: %d\n", length);
34
35 //Third task
36 const char* ptr2 = &my_string[0];
37
38 size_t length2 = strlen(ptr2);
39 printf("Length of string: %zx\n", length2);
40
41 //Forth task
42 char second_string[MY_SIZE] = "ELLO";
43 printf("Second string: %s\n", second_string);
44
45 strcpy(second_string, ptr2);
46 printf("Copied second string: %s\n", second_string);
47
48 //Fifth task
49 char string1[20] = "Hello";
50 const char string2[10] = ", world!";
51
52 printf("United strings: %s\n", strcat(string1, string2));
53
54 //Sixth task
55 char lh[5] = "abcd";
56 char rh[5] = "abcd";
57
58
59
60
61 const char *lhs = &lh[0];
62 const char *rhs = &rh[0];
63
64 printf("Result of comparison: %d\n", strcmp(lhs, rhs));
65
66 //Seventh task
67 unsigned char reg[15] = "AbCdEfGhI";
68
69 printf("Initial string: %s\n", reg);
70
71 for(int i = 0; i < 15; i++) {
72     reg[i] = tolower(reg[i]);
73 }
74 printf("Lowercase: %s\n", reg);
75
76 for(int i = 0; i < 15; i++) {
77     reg[i] = toupper(reg[i]);
78 }
79 printf("Uppercase: %s\n", reg);
80
81 return 0;
82 }

```

5. Результат выполнения программы.

```

Enter string:
BELL
Length of string: 4
Length of string: 4
Length of string: 4
Second string: ELLO
Copied second string: BELL
United strings: Hello, world!
Result of comparison: 0
Initial string: AbCdEfGhI
Lowercase: abcdefghi
Uppercase: ABCDEFGHI

Process returned 0 (0x0)   execution time : 1.448 s
Press any key to continue.

```

Задание 2.2

1. Постановка задачи.

Конвертируйте введенные заданные как строки: число с плавающей точкой (double) и целое число (int) в значения типа double и int, используя функциями atof и atoi. См. документацию по ссылке

<https://en.cppreference.com/w/c/string/byte>.

2. Математическая модель.

-

3. Список идентификаторов.

Имя	Тип	Смысл
a	char[5]	Вводимая строка
b	char[3]	Вводимая строка
*c	const char	Указатель на первый элемент строки a
*d	const char	Указатель на первый элемент строки b
e	double	Результат
f	int	Результат

4. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      char a[5] = "1.25";
7      char b[3] = "15";
8
9      const char *c = &a[0];
10     const char *d = &b[0];
11
12     double e = atof(c);
13     int f = atoi(d);
14
15     printf("%f\n", e);
16     printf("%d", f);
17
18     return 0;
19 }
20

```

5. Результат выполнения программы.

```

1.250000
15
Process returned 0 (0x0)   execution time : 0.115 s
Press any key to continue.

```

Задание 2.3

1. Постановка задачи.

Создайте строку от 10 до 20 символов, используя только цифры, латинский буквы в разных регистрах пробельные символы и символы

пунктуации. Организуйте цикл, где каждый символ подробно тестируется функциями типа `int is*(*... *)` (например — `isdigit`,

`ispunct`). См. документацию по ссылке

<https://en.cppreference.com/w/c/string/byte>. Оформите распечатку информации по каждому символу в виде списка на экране, чтобы можно было прочесть информацию о том что представляет из себя

4

каждый символ (своими словами, в свободной форме). Постарайтесь

использовать только латиницу.

2. Математическая модель.

-

3. Список идентификаторов.

Имя	Тип	Смысл
a	unsigned char[15]	Введённая строка
i	int	Счётчик цикла

4. Код программы.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <ctype.h>
4  #include <locale.h>
5
6  int main()
7  {
8      unsigned char a[15] = "Hello, 12345!!!";
9
10     for(int i = 0; i < 15; i++) {
11         if (isgraph(a[i])) {
12             if (isalnum(a[i])) {
13                 if (isalpha(a[i])) {
14                     if (islower(a[i])) printf("a[%d] is alphabetic lowercase\n", i);
15                     else printf("a[%d] is alphabetic uppercase\n", i);
16                 }
17                 else printf("a[%d] is digit\n", i);
18             }
19             else printf("a[%d] is punctuation character\n", i);
20         }
21         else if (isspace(a[i])) printf("a[%d] is space\n", i);
22     }
23
24     return 0;
25 }
26
```

5. Результат выполнения программы.

```
a[0] is alphabetic uppercase
a[1] is alphabetic lowercase
a[2] is alphabetic lowercase
a[3] is alphabetic uppercase
a[4] is digit
a[5] is punctuation character
a[6] is space
a[7] is digit
a[8] is digit
a[9] is digit
a[10] is digit
a[11] is digit
a[12] is punctuation character
a[13] is punctuation character
a[14] is punctuation character

Process returned 0 (0x0)   execution time : 0.156 s
Press any key to continue.
```


